UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants. Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2019-12, page [5]

Unofficial model answer:

a) Three characteristics of a Symmetric Multiprocessor (SMP): (1) Two or more identical processors (often called cores) share a single main memory and a common system bus or interconnect, so all processors have equal access to all memory and I/O devices. (2) All processors are peers (symmetric): each can execute any task, including operating-system code, and there is no master–slave relationship; the OS schedules work across all processors. (3) The system is tightly coupled and typically uses a single, shared copy of the operating system, with hardware cache coherence (e.g., snooping or directory protocols) maintaining consistency across per-processor caches. (Other acceptable points: uniform memory access (UMA) time, shared I/O, scalability limited by bus/memory contention.)

b) One availability advantage: because all processors are symmetric and can run any task, if one processor fails the remaining processors can continue to run the operating system and application workload, so the system degrades gracefully rather than failing completely. This gives higher availability/fault tolerance than a uniprocessor or an asymmetric master–slave multiprocessor, where failure of the master would halt the system.

Marking points:

• *1 mark: shared single main memory / common bus or interconnect with equal access for all processors*

• *1 mark: all processors are identical peers (symmetric), any processor can execute any task including OS code; no master–slave*

• *1 mark: tightly coupled single OS image and/or cache coherence / UMA characteristic*

• *1 mark: availability advantage stated — failure of one processor does not stop the system; remaining processors continue (graceful degradation)*

• *1 mark: explanation/contrast — because processors are symmetric and can run any task, the workload/OS can continue on surviving processors (unlike master–slave or uniprocessor)*

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied. Mark allocation is inferred from the 5-mark total (3 marks for part a, 2 marks for part b).

QUESTION 2 (5 marks)

Source: CSC429-2019-12, page [2]

Unofficial model answer:

a) The Von Neumann Architecture was first introduced in the **first generation** of computers (the stored-program concept, associated with the EDVAC design, c. 1945–1950s).

b) A labelled Von Neumann Architecture diagram should show the CPU (containing the Control Unit (CU), Arithmetic Logic Unit (ALU), and registers) connected to Main Memory, with Input and Output devices, all linked by a common bus/system bus. Data and instructions share the same memory and bus.

c) The main task of the Control Unit (CU) is to **fetch, decode and direct the execution of instructions**. It fetches instructions from main memory, decodes them, and generates the control signals that coordinate the ALU, registers, memory and I/O so that each instruction is carried out in the correct sequence. It does not itself perform arithmetic; it controls and synchronises the other components.

Marking points:

• *1 mark: (a) First generation (stored-program concept).*

© Hak Cipta Universiti Teknologi MARA

• 2 marks: (b) Correct diagram with CPU (CU + ALU + registers), main memory, input and output, connected by a common bus (1 mark for components, 1 mark for correct connections/labels).

• 2 marks: (c) CU fetches and decodes instructions and generates control signals to coordinate/direct other components (1 mark for fetch/decode, 1 mark for control/coordination of execution).

Model confidence: high.

Model note: Part (b) is a drawing question; the answer describes the required diagram since no figure is supplied. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 3 (5 marks)

Source: CSC429-2018-06, page [2]

Unofficial model answer:

a) Machine X is the **Difference Engine** (Charles Babbage's mechanical machine, later followed by his Analytical Engine). The clue 'solve four basic arithmetic functions' and 'improved from the Stepped Reckoner' points to Babbage's Difference Engine, the machine that is regarded as the first machine deserving to be called a computer.

b) The Difference Engine belongs to the **first generation** of computers (mechanical/electro-mechanical era, roughly 1642–1945), which covers early mechanical calculating devices such as the Pascaline, Stepped Reckoner and Babbage's engines.

c) The function of machine X is to **perform the four basic arithmetic operations (addition, subtraction, multiplication and division)** automatically by mechanical means, i.e. it was designed as an automatic mechanical calculator to compute and print mathematical/arithmetic tables.

d) The machine invented prior to machine X is the **Stepped Reckoner** (Leibniz's stepped reckoner), an early mechanical calculator that influenced Babbage's design; the Pascaline (Pascal's calculator) is also an acceptable earlier mechanical calculator.

e) Because of his success in conceiving machine X, Charles Babbage is given the title **'Father of the Computer'**.

Marking points:

• 1 mark (a): Correctly names the Difference Engine (accept Analytical Engine only if clearly justified; Difference Engine is the expected answer).

• 1 mark (b): States first generation (mechanical era).

• 1 mark (c): States it performs the four basic arithmetic operations / automatic mechanical calculation.

• 1 mark (d): Names the Stepped Reckoner (accept Pascaline as an earlier mechanical calculator).

• 1 mark (e): States 'Father of the Computer'.

Model confidence: medium.

Model note: Figure 1 is supplied only as a caption/extracted text; the answer relies on the statements in the question text. The question wording ('solve four basic arithmetic functions', 'improved from Stepped Reckoner') is slightly loose historically, since the Difference Engine was designed for tabulating polynomials; the expected exam answer is nonetheless the Difference Engine. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 4 (5 marks)

Source: CSC429-2026-02, page [3]

Unofficial model answer:

The equation is interpreted as computing (-56 decimal) + (A3 hexadecimal). A3H = 10×16 + 3 = 163 decimal. The sum is -56 + 163 = 107 decimal. Using 8-bit two's complement arithmetic: 56 decimal = 00111000_2. 1's complement of 56 = 11000111_2. 2's complement of 56 = 11001000_2 (C8H). A3H = 10100011_2. 1's complement method: add the 1's complement of 56 to A3H: 11000111 + 10100011 = 1 01101010. Since there is a carry out of the sign bit, perform end-around carry: 01101010 + 1 = 01101011_2 = 6BH = 107 decimal. 2's complement

© Hak Cipta Universiti Teknologi MARA

method: add the 2's complement of 56 to A3H: 11001000 + 10100011 = 1 01101011. Discard the carry out of the sign bit; the result is 01101011_2 = 6BH = 107 decimal. Both methods give the same positive result, 107 decimal (6BH), which is correct because the magnitude of the positive operand (163) exceeds the magnitude of the negative operand (56).

Marking points:

• 1 mark: Correctly convert -56 to 8-bit 1's complement (11000111) and 2's complement (11001000), and A3H to binary (10100011).

• 1 mark: Perform 1's complement addition: 11000111 + 10100011 = 1 01101010, showing the carry out.

• 1 mark: Apply end-around carry in 1's complement: 01101010 + 1 = 01101011, and interpret as +107 (6BH).

• 1 mark: Perform 2's complement addition: 11001000 + 10100011 = 1 01101011, discard carry, result 01101011 = +107 (6BH).

• 1 mark: State final answer clearly (107 decimal or 6BH) and note that both methods agree and the result is positive.

Model confidence: high.

Model note: Assumes 8-bit word length and that the equation means (-56) + A3H. The original question text '-56s + A3H' is interpreted as '-56 decimal plus A3 hexadecimal'; the 's' is likely a typo or subscript. No figure or table was provided.

QUESTION 5 (10 marks)

Source: CSC429-2025-02, page [4]

Unofficial model answer:

a) In interrupt-driven I/O, the CPU initiates each I/O transfer by issuing a command to the device controller and then continues executing other instructions. When the device completes the transfer (or needs service), it raises an interrupt request; the CPU suspends its current program, saves state, and runs an interrupt service routine (ISR) to transfer data or handle status. Thus the CPU is involved in every data transfer (or at least every block) and must execute the ISR, but it does not busy-wait. In Direct Memory Access (DMA), the CPU programs the DMA controller with the source/destination addresses and transfer count, then the DMA controller performs the entire block transfer directly between the device and memory without CPU intervention. The CPU is interrupted only once per block (at completion or error), so CPU involvement is limited to setup and final completion handling, freeing the CPU for other work during the transfer.

b) An interrupt is a signal (hardware or software) that causes the CPU to temporarily suspend the execution of the current instruction stream, save the current context, and transfer control to a designated interrupt handler (ISR). After the handler completes, the saved context is restored and the interrupted program resumes. Interrupts are used to notify the CPU of asynchronous events such as I/O completion, timer expiry, or errors.

c) During a context switch, the operating system saves the state (context) of the currently running process and restores the state of the next process to be run. The steps typically include: (1) a triggering event (interrupt, trap, or system call) causes the CPU to enter kernel mode; (2) the kernel saves the current process's context—program counter, general-purpose registers, status flags, and other processor state—into its process control block (PCB); (3) the scheduler selects the next process; (4) the kernel restores the selected process's context from its PCB, including PC and registers; (5) the CPU resumes execution of the new process. The switch involves saving/restoring state and may also involve memory-management updates (e.g., page-table base register) and cache/TLB effects.

d) The benefit of a context switch is that it enables multitasking and CPU sharing: the CPU can be switched from a blocked or lower-priority process to another ready process, improving CPU utilization, system responsiveness, and fairness. It allows the OS to implement time-sharing, priority scheduling, and to overlap I/O waits with useful computation, so that the system does not idle while one process waits for I/O or an event.

Marking points:

• 1 mark: Interrupt-driven I/O: CPU issues command and continues; device interrupts on completion; CPU executes ISR for each transfer/block.

© Hak Cipta Universiti Teknologi MARA

• 1 mark: DMA: CPU programs DMA controller (addresses, count) and is interrupted only at block completion; DMA transfers data directly to/from memory without per-byte CPU involvement.

• 1 mark: Clear contrast that CPU involvement is per-transfer in interrupt-driven I/O versus setup/completion only in DMA.

• 1 mark: Definition of interrupt as a signal/event that suspends current execution and transfers control to an ISR.

• 1 mark: Mention of saving context and resuming after ISR (or equivalent).

• 1 mark: Context switch: save current process context (PC, registers, flags) into PCB.

• 1 mark: Context switch: select next process and restore its context from PCB.

• 1 mark: Context switch: resume execution of new process; mention kernel mode/scheduler involvement.

• 1 mark: Benefit: enables multitasking/time-sharing and improves CPU utilization/responsiveness/fairness.

• 1 mark: Benefit: allows overlap of I/O waits with computation or switching away from blocked processes.

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. The question text says 'Direct Manipulation Access' but the standard term is Direct Memory Access (DMA); the answer assumes DMA. No figure or table was supplied.

QUESTION 6 (10 marks)

Source: CSC429-2016-06, page [5]

Unofficial model answer:

a) Direct mapping does not need a replacement algorithm. In direct mapping each main-memory block maps to exactly one cache line (line = block number mod number of lines), so when a new block is brought in there is only one possible line it can occupy; that line is simply overwritten. No choice among candidate lines arises, hence no replacement policy is required. (Fully associative and set-associative caches do require a replacement algorithm because a new block may be placed in any line of the cache or of its set.)

b) Four replacement algorithms:

1. Random replacement: a line is chosen at random from the candidate lines (the whole cache for fully associative, or the target set for set-associative). It is simple and cheap in hardware, requires no bookkeeping, but gives no locality-based performance guarantee.
2. FIFO (First-In, First-Out): the line that has been resident in the cache the longest is replaced. It is implemented with a circular queue/age counter per line; it is simple but can evict a heavily used line simply because it is old.
3. LRU (Least Recently Used): the line that has not been referenced for the longest time is replaced, exploiting temporal locality. It gives good hit ratios but needs recency bookkeeping (e.g., counters or a reference/use bit matrix), which is costly for high associativity.
4. LFU (Least Frequently Used): the line with the smallest number of references (access count) is replaced. It favours frequently used blocks but suffers from stale counts and does not adapt quickly to changing working sets; counters must be maintained per line.

(Other acceptable algorithms include Optimal/MIN, NRU, and Second-Chance/Clock; any four correctly described policies earn full credit.)

Marking points:

• 1 mark: (a) correctly identifies direct mapping as the mapping function that needs no replacement algorithm.

• 1 mark: (a) gives a valid reason (each block maps to exactly one line, so the line is simply overwritten).

• 1 mark: (b) names and correctly describes Random replacement.

• 1 mark: (b) names and correctly describes FIFO replacement.

• 1 mark: (b) names and correctly describes LRU replacement.

• 1 mark: (b) names and correctly describes LFU replacement (or another valid algorithm such as Optimal, NRU, Second-Chance).

• 2 marks: overall accuracy/terminology and completeness across the four algorithms (e.g., each description states the selection rule and a brief property).

© Hak Cipta Universiti Teknologi MARA

• 2 marks: clarity and structure of the answer, with each algorithm clearly separated and no conceptual errors (e.g., confusing mapping with replacement).

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question text does not include a figure or table, so no figure-based limitation applies. Mark allocation is inferred because the paper only states the total (10 marks); the split between (a) and (b) and the weighting of descriptions are assumed.

QUESTION 7 (10 marks)

Source: CSC429-2024-01, page [3]

Unofficial model answer:

a) Convert -156.15625_{10} to IEEE 754 single precision (hex). Step 1 — sign: the number is negative, so S = 1. Step 2 — magnitude in binary: 156 = 128 + 16 + 8 + 4 = 10011100_2. The fraction 0.15625 = 5/32 = 1/8 + 1/32 = 0.00101_2. So $156.15625_{10} = 10011100.00101_2$. Step 3 — normalise: $10011100.00101_2 = 1.001110000101_2 \times 2^7$ (shift the point 7 places left). Step 4 — exponent: E = 7 + 127 = 134 = 10000110_2. Step 5 — mantissa (23 bits, drop the leading 1): 001110000101 followed by zeros = 0011 1000 0101 0000 0000 000. Step 6 — assemble: S | E | M = 1 10000110 00111000010100000000000. Group into nibbles: 1100 0011 0001 1100 0010 1000 0000 0000 = C31C2800H. Answer: **C31C2800H**.

b) Convert BD40 0000H to decimal. Step 1 — expand to binary: BD400000H = 1011 1101 0100 0000 0000 0000 0000 0000. Step 2 — split fields: S = 1 (negative); E = 0111 1010_2 = 122; M = 100 0000 0000 0000 0000 0000_2. Step 3 — since 0 < E < 255 the number is normalised; unbiased exponent = E − 127 = 122 − 127 = −5. Step 4 — significand = 1.M = 1.1_2 = 1.5. Step 5 — value = $(-1)^1 \times 1.5 \times 2^{-5} = -1.5/32 = -0.046875$. Answer: **$-0.046875_{10}$**.

Marking points:

• a) 1 mark: correct sign bit S = 1 for the negative number

• a) 1 mark: correct binary of integer part 156 = 10011100_2

• a) 1 mark: correct binary of fraction 0.15625 = 0.00101_2

• a) 1 mark: correct normalisation $1.001110000101_2 \times 2^7$ and biased exponent 7 + 127 = 134 = 10000110_2

• a) 1 mark: correct 23-bit mantissa 00111000010100000000000 and final hex C31C2800H

• b) 1 mark: correct binary expansion of BD400000H and field split (S=1, E=01111010_2=122, M=100…0)

• b) 1 mark: correct unbiased exponent 122 − 127 = −5 and significand 1.1_2 = 1.5

• b) 1 mark: correct final decimal value −0.046875 (accept -1.5×2^{-5} or −3/64)

• Overall: 1 mark for coherent, clearly shown workflow in both parts (binary/normalisation steps visible)

Model confidence: high.

Model note: Unofficial model answer, not an official UTM marking scheme. The question text shows '-156.1562510' and 'BD40 0000H'; the trailing '10' is read as the base-10 subscript, i.e. -156.15625_{10}. No figure or table was supplied. Mark allocation across the 10 marks is indicative.

QUESTION 8 (10 marks)

Source: CSC429-2014-12, page [4]

Unofficial model answer:

a) DMA (Direct Memory Access) is a hardware-controlled I/O technique in which a dedicated DMA controller transfers blocks of data directly between an I/O device and main memory without the CPU executing a load/store instruction for each word. The CPU programs the DMA controller with the source/destination addresses, transfer count and direction, then the controller performs the transfer and raises an interrupt when complete.

b) DMA improves overall performance because the CPU is freed from per-word data movement, so it can execute other instructions while the DMA controller handles the block transfer. This reduces CPU overhead

© Hak Cipta Universiti Teknologi MARA

and instruction count, increases effective throughput for large block I/O, and allows memory and I/O transfers to overlap with computation (cycle stealing or burst mode), improving system utilisation and response time.

c) Two major drawbacks of programmed and interrupt-driven I/O that motivated DMA: (1) Programmed I/O is CPU-intensive and wasteful: the CPU repeatedly polls the device status and moves each word itself, so it is busy-waiting and cannot do useful work during the transfer. (2) Interrupt-driven I/O still incurs a context switch/interrupt overhead for every word or small block, so for high-speed or large-block devices the interrupt rate and per-word CPU involvement become prohibitive, limiting throughput and consuming CPU time. (Other acceptable: both tie up the CPU for each data item and are unsuitable for high-speed devices.)

d) Redrawn DMA architecture: the CPU, primary memory and DMA controller are connected by the system bus; the DMA controller is also connected to the I/O device/controller. Data path for a block transfer is Device/Controller <-> DMA controller <-> Primary Memory, with the CPU using the bus for control/status and being interrupted on completion. Compared with Figure 2 (CPU <-> Controller <-> Device, with memory accessed only via the CPU), the DMA controller is inserted as a bus master between the device controller and memory, so data can flow directly to/from memory without passing through the CPU. (Figure 2 itself was not supplied beyond its caption, so the redraw is described rather than drawn.)

Marking points:

• *1 mark: DMA definition — direct transfer between I/O device and memory without CPU per-word involvement*
• *1 mark: DMA controller programmed with address/count/direction and interrupts CPU on completion*
• *1 mark: performance improvement — CPU freed from data movement / overlap of I/O with computation*
• *1 mark: performance improvement — reduced CPU overhead/instruction count, higher throughput for block transfers*
• *1 mark: drawback 1 — programmed I/O busy-waiting/polling wastes CPU cycles*
• *1 mark: drawback 1 explanation — CPU moves each word itself, cannot do useful work during transfer*
• *1 mark: drawback 2 — interrupt-driven I/O has per-word interrupt/context-switch overhead*
• *1 mark: drawback 2 explanation — high interrupt rate for high-speed/large-block devices limits throughput*
• *1 mark: redrawn diagram shows DMA controller between device controller and memory on system bus*
• *1 mark: diagram shows direct device-controller-to-memory data path and CPU not in data path (CPU control/interrupt only)*

Model confidence: medium.

Model note: Figure 2 was available only by caption/extracted text, so part (d) is answered by describing the required DMA architecture rather than reproducing the original drawing. Mark allocation is an unofficial estimate consistent with the 10-mark total.

QUESTION 9 (10 marks)

Source: CSC429-2016-12, page [3]

Unofficial model answer:

a) Architecture vs organization: Computer architecture refers to the attributes of a computer system that are visible to a programmer, i.e., the conceptual design and functional behaviour of the system. It includes the instruction set architecture (ISA), data types, addressing modes, registers visible to the programmer, and I/O mechanisms. Architecture answers 'what the computer does' from the programmer's perspective. Computer organization refers to the operational units and their interconnections that realize the architectural specifications. It deals with the internal hardware details that are transparent to the programmer, such as control signals, memory technology, bus structure, and the physical implementation of the CPU. Organization answers 'how the computer implements the architecture'. For example, whether a computer has a multiply instruction is an architectural issue, while whether that multiply is implemented by a dedicated hardware multiplier or by repeated addition is an organizational issue. Another example: the programmer-visible register set is architecture, while the technology and arrangement of the ALU and control unit are organization. In short, architecture is the programmer-visible functional view, and organization is the implementation-level structural view.

© Hak Cipta Universiti Teknologi MARA

b) Three CPU components and their functions: (1) Control Unit (CU): The control unit directs the operation of the CPU and the rest of the computer. It fetches and decodes instructions, then generates the timing and control signals that cause the other components (ALU, registers, memory, I/O) to perform the required operations. It coordinates the fetch-decode-execute cycle and ensures that data moves correctly between units. (2) Arithmetic Logic Unit (ALU): The ALU performs arithmetic operations such as addition, subtraction, multiplication, and division, and logical operations such as AND, OR, NOT, XOR, as well as comparisons and shifts. It takes operands from registers, performs the requested operation, and places the result in a register while setting condition flags (e.g., zero, carry, sign, overflow). (3) Registers: Registers are small, high-speed storage locations inside the CPU. They hold operands, results, addresses, and control information during instruction execution. Key registers include the Program Counter (PC), which holds the address of the next instruction; the Instruction Register (IR), which holds the current instruction being executed; the Memory Address Register (MAR) and Memory Data Register (MDR) for memory transfers; and general-purpose registers for temporary data. Registers provide the fastest storage in the memory hierarchy and reduce the number of slow memory accesses. (Other acceptable components include the internal buses, cache, and instruction decoder, but the three above are the standard expected answers.)

Marking points:

• 1 mark: Correct definition of architecture as programmer-visible attributes (ISA, data types, addressing modes, etc.).

• 1 mark: Correct definition of organization as implementation-level structure (control signals, buses, technology, etc.) that realizes the architecture.

• 1 mark: Clear distinction with at least one example (e.g., multiply instruction vs hardware multiplier).

• 1 mark: Identification of Control Unit (CU) as a CPU component.

• 1 mark: Explanation of CU function (fetch/decode, generate control signals, coordinate execution).

• 1 mark: Identification of Arithmetic Logic Unit (ALU) as a CPU component.

• 1 mark: Explanation of ALU function (arithmetic and logical operations, flags).

• 1 mark: Identification of registers (or another valid CPU component) as a CPU component.

• 1 mark: Explanation of register function (high-speed temporary storage, PC/IR/MAR/MDR roles).

• 1 mark: Overall clarity, correct terminology, and completeness of the answer.

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify which three CPU components to discuss; the answer uses the Control Unit, ALU, and registers, which are the most standard choices. Other valid components (e.g., instruction decoder, internal buses, cache) could be accepted if explained correctly. Mark allocation is indicative and assumes a 10-mark total with roughly 4 marks for part (a) and 6 marks for part (b).

QUESTION 10 (10 marks)

Source: CSC429-2019-06, page [4, 5]

Unofficial model answer:

a) Access methods: (i) Cassette tape — sequential access (data read in order along the tape; to reach a later track the tape must be wound forward). (ii) Floppy disk — direct (random) access: the read/write head moves to the required track and the disk rotates to the sector, so blocks can be reached without reading everything before them. (iii) Cache memory — random access: any cache line is addressed directly by its tag/index and accessed in essentially constant time. (iv) RAM (main memory) — random access: any word is addressed directly via its address, independent of location. Note: the question's sub-labels (i)–(iv) are not visible in the extracted text; the mapping above follows the order of the four scenarios (tape, floppy, cache, RAM).

b) SRAM (Static RAM): built from flip-flop (typically 6-transistor) cells that hold a bit as long as power is supplied; no refresh needed; very fast (access times of a few nanoseconds); low density and high cost per bit, so used for cache. DRAM (Dynamic RAM): stores each bit as charge on a capacitor (1 transistor + 1 capacitor per cell); charge leaks, so it must be periodically refreshed (every few milliseconds); slower than SRAM but

© Hak Cipta Universiti Teknologi MARA

denser and cheaper per bit, so used for main memory.

c) In a three-level cache (L1, L2, L3), L1 is the fastest — it is smallest and closest to the CPU core, with the lowest access latency. Cache is implemented using SRAM, because SRAM's speed and lack of refresh make it suitable for the small, fast cache arrays.

Marking points:

- *1 mark: (i) cassette tape identified as sequential access*
- *1 mark: (ii) floppy disk identified as direct/random access*
- *1 mark: (iii) cache identified as random access*
- *1 mark: (iv) RAM identified as random access*
- *1 mark: SRAM described as flip-flop based, no refresh, fast, low density/high cost (cache use)*
- *1 mark: DRAM described as capacitor-based, needs refresh, slower, high density/low cost (main memory use)*
- *1 mark: SRAM vs DRAM comparison (speed/cost/density/refresh) clearly stated*
- *1 mark: L1 identified as the fastest cache level*
- *1 mark: SRAM identified as the RAM type used for cache*
- *1 mark: justification linking L1 closeness/small size to lowest latency and/or SRAM speed to cache use*

Model confidence: medium.

Model note: The sub-labels (i)–(iv) in part (a) are not present in the extracted question text; the mapping assumes the order of the four scenarios (tape, floppy, cache, RAM). This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2023-07, page [6]

Unofficial model answer:

Figure 3 represents the expression 242 − (B + AC), i.e. 242 minus the sum of register B and the product of registers A and C. Registers A, B and C are read-only; x, y and z are temporaries; the final result must end in register y. The three programs below use only the mnemonics listed in Table 1.

Zero-address program: With a stack machine, operands are pushed and the arithmetic instructions pop their operands and push the result. The constant 242 is pushed first, then B, then A and C are pushed and MUL is executed to leave AC on the stack; ADD then combines B and AC; SUB computes 242 − (B + AC); finally pop stores the top of stack into y. Program: PUSH 242; PUSH B; PUSH A; PUSH C; MUL; ADD; SUB; POP y. Comments: each PUSH places an operand on the stack; MUL replaces A and C by A*C; ADD replaces B and A*C by B+AC; SUB replaces 242 and (B+AC) by 242−(B+AC); POP y writes the result to y.

One-address program: A one-address machine uses an implicit accumulator. LOAD copies a memory/register operand into the accumulator; ADD, SUB, MUL and DIV combine the accumulator with the named operand and leave the result in the accumulator; STOR copies the accumulator to the named destination. Because A, B and C cannot be modified, the product AC is formed in a temporary: LOAD A; MUL C; STOR x (x = A*C); LOAD B; ADD x (accumulator = B + AC); STOR z (z = B + AC); LOAD 242; SUB z (accumulator = 242 − (B + AC)); STOR y. Comments: the first three instructions compute AC into x; the next two compute B + AC into z; the last three compute the final subtraction and store the answer in y.

Three-address program: Each three-address instruction has the form OP dest, src1, src2 and does not destroy its sources. Using temporaries x, y and z: MUL x, A, C (x = A*C); ADD z, B, x (z = B + AC); SUB y, 242, z (y = 242 − (B + AC)). Comments: MUL forms the product without changing A or C; ADD forms the parenthesised sum without changing B; SUB performs the final subtraction and places the required final answer directly in y. No STOR is needed because the destination of the last instruction is y.

Marking points:

• 2 marks: correctly identifies the expression as 242 − (B + AC) and respects the read-only constraint on A, B, C (uses only x, y, z as temporaries).

• 6 marks: zero-address program — correct use of PUSH/POP and stack order (2), correct MUL for AC (1), correct ADD for B+AC (1), correct SUB order 242 − (B+AC) (1), final POP y (1).

• 6 marks: one-address program — correct LOAD/STOR/accumulator usage (2), computes AC into a temporary without modifying A or C (1), computes B+AC (1), computes 242 − (B+AC) in the right order (1), final STOR y (1).

• 4 marks: three-address program — MUL x, A, C (1), ADD z, B, x (1), SUB y, 242, z (1), final answer in y and no source registers modified (1).

• 2 marks: correct assembly notation and meaningful comments throughout (award proportionally across the three programs).

Model confidence: high.

Model note: Figure 3 is supplied only as extracted text '242 - (B + AC)'; the answer assumes this is the intended expression. Table 1 lists only mnemonics, not operand syntax, so standard zero-, one- and three-address operand forms are assumed. The one-address column in the extracted table omits MUL; the answer assumes MUL is available on the one-address machine because the question requires computing AC. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA